```
//user.proto
syntax="proto3";
package Kuwo;
option cc_generic_services=true;// 生成通用服务接口（仅在需要时）

// 结果代码
message ResultCode{
  int32 errcode = 1; // 错误码，0表示成功
  bytes errmsg = 2; // 错误消息

// 用户模型
message User{
  bytes name = 1;  // 用户名
  bytes pwd = 2;   // 密码

// 服务端响应登录结果
message LoginResponse{
  ResultCode result = 1; // 包含错误码和信息
  bool success = 2;      // 成功标志位

// 注册请求
message RegisterRequest{
  uint32 id = 1;  // 用户 id
  bytes name = 2;
  bytes pwd = 3;

// 注册成功返回响应
message RegisterResponse{
  ResultCode result = 1;
  bool success = 2;

// 函数模型
// 生成通信协议类型UserServiceRpc_Stub
google::protobuf::service
service UserServiceRpc
  // 定义login函数方法
  // 客户端发送LoginRequest 消息，服务端回复 LoginResponse
  rpc Login(LoginRequest) returns(LoginResponse);
  // 定义Register函数方法
  // 客户端发送注册信息，服务端返回注册结果
  rpc Register(RegisterRequest) returns(RegisterResponse);
```

package Kuwo; 定义命名空间，在生成的 C++ 代码中，所有的类都会被包裹在 namespace Kuwo { ... } 里，有助于防止命名冲突

关键词：这告诉编译器生成继承自 google::protobuf::Service 的类。

工作原理：当proto文件包含service时，编译器会生成两个类：UserServiceRpc和UserServiceRpc_Stub。简单的理解一个抽象类，用于服务端，服务端继承这个抽象类并重写Login方法；而客用于客户端，客户端实例化这个Stub对象，并传入一个RpcChannel对象后，就可以在服务端的后面通信发来调用服务端的Login方法了。 虽然Implement作为之的Login的方法，一样的，具体来说：
（1）对于服务端，需要继承Stub这个抽象类并实现重写Login函数，要求客户端发送LoginRequest类型消息及从Rpc收到LoginResponse类型消息；
（2）对于客户端，客户端只需要直接实例化UserServiceRpc_Stub并传入一个 RpcChannel 对象（这个 Channel 负责网络的 TCP 连接管理和数据发送），就可以直接在本地调用这个服务端的Login方法，比如实例化好服务Stub，并调用通道之stub.Login()就可以调用远程方法了。
注：
（1）由于这里proto文件内生成的代码：
因此UserServiceRpc类自动继承自google::protobuf::Service，它需要一个 CallMethod 接口，框架层（比如即将实现的 RPC Provider）会调用这个函数，在调用这个 CallMethod，它内部会根据函数名称查看下的 Login 函数，
（2）但客户端Stub的函数内部会调用它的父类google::protobuf::RpcChannel的 CallMethod 方法，这是一个虚函数，需要实现 RpcChannel::CallMethod()。CallMethod内部会发送一条网络请求，
这样调用的完整逻辑：
客户端调用：stub.Login()
在框架层：Stub 自动调用 Channel->CallMethod
网络传输：Channel 将数据序列化，通过 asio 发送给服务器
服务端分发：服务器收到后，根据方法名调用对应的 UserServiceRpc
找到对象：服务器在 map 中找到对应类型下正确的那个活生生对象
执行业务：调用你重写的 Login 函数，完成逻辑处理
原路返回：通过 Channel->SendRpcResponse() 把结果再沿回网络

```
// //rpcheader.proto
// syntax="proto3"
// package Kuwo;
// // 用于RPC调用的
// message RpcHeader{
//   bytes service_name=1;  // 服务名
//   bytes method_name=2;   // 方法名
//   uint32 args_size=3;    // 参数大
// }
```

假设客户端发起了一个 Login 请求，发 1 个 数据消息，这两个文件之间的关系和流程体现在代码：
1. 第一阶段：解析 RpcHeader（服务寻址）
客户端发送的网络流：包 size1 结构序字头后，数据 RpcHeader 进行反序列化，
service_name：框架通过它去 service_map 查找对应的 UserServiceRpc 对象。
method_name：框架通过它去该 Service 的描述符查找对应 Login 方法。
args_size：框架知道接下来的二进制长度，是为 protobuf 反序列化参数的，从而知道边界。
此时的状态：框架已经知道要调用一个名叫 login 的方法，但还不知道该方法的参数是什么。
2. 第二阶段：解析 user.proto 定义的参数（业务解析）
框架根据上一步获取的 service_name，查找注册时存下的 UserServiceRpc 对象。
反序列化：框架根据 user.proto 生成的类，将剩余字节流反序列化为 LoginRequest 对象。
执行调用：框架调用重写的 Login(request, response) 函数。
业务处理：你从 request 里读出 name 和 pwd。
此时的状态：你正在处理业务逻辑（比如查数据库）。
总的来说：Kuwo.header.proto 是不需要编译的，解决“数据怎 size 和 type，怎么找对应的服”；
user.proto 则是不会自动调用，解决“数据内容是 name 和 pwd 的结构”。这两个文件，没有需要修改数据啊 。

注：项目文件中的红色字体所标记的函数是RPC框架中相应部分所提供的C++语言通用API接，并非是自己定义的实现具函数或标准库函数

注意，RPC的服务端并不是由一个函数来调用，而是由“框架层”，“业务层”两部分构成的，客户端的发起的服务请求处理逻辑是由框架层来解决业务层无关的，因此前端提到的“服务端需要继承UserServiceRpc类”，准确来说，应该是框架端的业务层实现需要继承UserServiceRpc类，由于服务端是业务主体**产生数据**，需要先处理某些业务，因此框架层的代码会先处理无需要业务逻辑的部分，往往在中途遇到需要数据时，又现身去的目的，比如说在中间会NotifyApplicationRpc告知服务的进程调用NotifyServer的函数processRpc()Service”，

服务端框架层与业务层之间的关系可以概括为：

至于框架层，本项目主要包括如下几个模块：
- Logger日志模块：实现log()函数，一条写盘，可重命名CC一行日志；
- conf配置模块：读取配置文件，调取配置文件详解的；
- RpcHeader模块：序列化 服务名 及 服务端的功能化操作；
- Krpc服务核心类；
- ZookeeperUtil封装ZooKeeper客户端

Logger日志模块

- **时间log —— 高性能、轻量级的C++日志库**
- 高性能：glog采用了**高效的缓冲机制**，减小了日志记录时程序执行性能的影响
- 简单易用：glog**接口简洁**，易于集成和使用
- 可定制性：glog支持**多种日志格式**，可以根据需求灵活选择的记录的日志内容
- **Krpc-Logger glog封装**

```
> KrpcLogger
  #include <glog/logging.h>
  #include <string>
// 基于glog的日志模块，封装 Google glog 日志系统
class KrpcLogger {
public:
    // 构造函数：启动初始化 glog
    explicit KrpcLogger(const char *argv0){
        google::InitGoogleLogging(argv0);
        FLAGS_colorlogtostderr = true;      // 启用色彩日志
        FLAGS_logtostderr = true;           // 默认日志标准错误
    }
    ~KrpcLogger(){
        google::ShutdownGoogleLogging();
    }
    // 提供静态日志接口方法
    static void Info(const std::string & message){
        LOG(INFO) << message;
    }
    static void Warning(const std::string & message){
        LOG(WARNING) << message;
    }
    static void Error(const std::string & message){
        LOG(ERROR) << message;
    }
    static void Fatal(const std::string & message){
        LOG(FATAL) << message;
    }
private:
    // 禁用拷贝构造和赋值运算符
    KrpcLogger(const KrpcLogger& ) = delete;
    KrpcLogger& operator=(const KrpcLogger& ) = delete;
};
```

Config配置模块

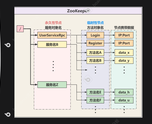
```
// // krpc 配置类
class KrpcConfig {
public:
    void LoadConfigFile(const char* config_file);  // 加载配置文件
    std::string Load(const std::string & key);     // 查询 key 对应的 value
private:
    void Trim(std::string & read_buf);             // 去掉字符串前后的空格
private:
    std::unordered_map<std::string, std::string> config_map; // 配置项键值映
};
```

Krpc服务端核心类

```
class KrpcProvider {
public:
    // 注册服务对象及其方法到 service_map，某种RPC方法的函数接口
    void NotifyService(google::protobuf::Service* service);
    // 样例函数
    ~KrpcProvider();
    // 启动RPC服务节点，开始提供RPC远程网络调用服务
    void Run();
private:
    // 服务信息结构体 存储服务对象和方法map
    // 一服务对象，方法一方法名，方法描述->
    struct ServiceInfo{
        google::protobuf::Service* service;  // 服务对象
        // 存储方法的 method_map 方法名，方法描述->
        std::unordered_map<std::string, const google::protobuf::MethodDescrip
    };
    // 连接回调函数：处理客户端连接事件
    void OnConnection(const muduo::net::TcpConnectionPtr& conn);
    // 消息回调函数：处理客户端发送的RPC请求
    void OnMessage(const muduo::net::TcpConnectionPtr& conn,
                   muduo::net::Buffer* buffer, muduo::Timestamp receive_time
    // 响应回调函数 发送 rpc 响应给客户端
    void SendRpcResponse(const muduo::net::TcpConnectionPtr& conn, google::p
private:
    // 事件循环
    muduo::net::EventLoop event_loop;
    // 服务map 保存服务对象和 服务信息 <service_name, service_info>
    std::unordered_map<std::string, ServiceInfo> service_map;
};
```

void KrpcProvider::Run() {
    // 获取配置文件单例 KrpcConfig 实例 (假设)
    std::string ip = KrpcConfig::GetInstance().GetConfig().Load("rpcserverip");
    int port = 0;
    try {
        port = std::stoi(KrpcConfig::GetConfig().Load("rpcserverport"));
    } catch (...) {
        // 解析失败，使用默认端口
        port = 8000;
    }
    // 设置监听地址
    muduo::net::InetAddress address(ip, port);
    // 创建 TcpServer 对象
    std::shared_ptr<muduo::net::TcpServer> server =
std::make_shared<muduo::net::TcpServer>(&event_loop, address, "KrpcProvider");
    // 绑定连接回调和消息回调，以实现网络连接和数据接收
    server->setConnectionCallback(std::bind(&KrpcProvider::OnConnection, this,
std::placeholders::_1));
    server->setMessageCallback(std::bind(&KrpcProvider::OnMessage, this,
std::placeholders::_1, std::placeholders::_2, std::placeholders::_3));
    // 设置muduo的线程数量
    server->setThreadNum(4);
    /**
    * @brief zookeeper 服务注册：将 service_name 和 method_name 注册到 zookeeper 服务器
    */
    // 开始监听RPC请求之前，先向ZooKeeper注册服务 Zookeeper 上，以 RPC 客户端可以从 Zookeeper 上发现服务
    ZkClient zkclient;
    zkclient.Start(); // 连接zookeeper 服务器
    // server_name 为命名空间，method_name 为临时节点
    for auto kpp : service_map {
        // service_name 在 zookeeper 中的目录路 "/" + service_name (永久节点)
        std::string service_path = "/" + kpp.first;
        char data[128] = {0};
        int len = sprintf(data, "%s:%d", ip.c_str(), port);
        // method_name 在 zookeeper 中的目录路 "/" + service_name/method_name (临时节点)
        auto const& methods = kpp.second.m_method_map;
        for(auto mp : methods) {
            std::string method_path = service_path + "/" + mp.first;
            char method_path_data[128] = {0};
            sprintf(method_path_data, "%s:%d", ip.c_str(), port);
            // ZOO_EPHEMERAL 表示这个节点是临时节点，会客户端断开连接后，Zookeeper会自动删除这个
            int flag = ZOO_EPHEMERAL;
            int state = zkclient.Create(method_path.c_str(), method_path_data,
strlen(method_path_data), ZOO_EPHEMERAL);
        }
    }
    // RPC 服务端准备启动，打印信息
    std::cout << "RpcProvider start service at ip: " << ip << ":" << port <<
std::endl;
    // 启动网络服务
    server->start();
    event_loop.loop(); // 阻塞事件循环
}

Run()是由RPC服务节点
1. 读取配置文件 (RPC 服务器 IP 和 端口)；
2. Muduo引擎实例化服务器监听和连接回调函数 server;
3. Zookeeper服务注册(kv store);
4. 启动事件循环 loop



void KrpcProvider::OnConnection(muduo::net::TcpConnectionPtr& conn) {
    // 检查连接是否建立
    if (conn->connected()) {
        // 连接建立，记录日志
        LOG(INFO) << "client connected: " << conn->peerAddress().toIpPort()
<< " -> " << conn->localAddress().toIpPort();
    } else {
        // 连接断开，记录日志
        LOG(INFO) << "client disconnected: " << conn->peerAddress().toIpPort()
<< " -> " << conn->localAddress().toIpPort();
    }
}

void KrpcProvider::OnMessage(const muduo::net::TcpConnectionPtr& conn,
                             muduo::net::Buffer* buffer,
                             muduo::Timestamp receive_time) {
    // 从缓冲区读取所有数据
    std::string recv_buf = buffer->retrieveAllAsString();
    LOG(INFO) << "received data, size: " << recv_buf.size();
    // 创建 CodedInputStream 用于解析数据
    google::protobuf::io::ArrayInputStream array_in(recv_buf.data(), recv_buf.
size());
    google::protobuf::io::CodedInputStream coded_in(&array_in);
    // 解析 header_size
    uint32_t header_size = 0;
    bool read_ok = coded_in.ReadVarint32(&header_size);
    if (!read_ok) {
        LOG(ERROR) << "failed to read header_size";
        return;
    }
    // 解析 RpcHeader
    std::string rpc_header_str;
    int limit = coded_in.PushLimit(header_size);
    read_ok = coded_in.ReadString(&rpc_header_str, header_size);
    if (!read_ok) {
        LOG(ERROR) << "failed to read rpc_header_str";
        return;
    }
    coded_in.PopLimit(limit);
    RpcHeader::RpcHeader rpc_header;
    if (!rpc_header.ParseFromString(rpc_header_str)) {
        LOG(ERROR) << "failed to parse rpc_header";
        return;
    }
    std::string service_name = rpc_header.service_name();
    std::string method_name = rpc_header.method_name();
    int args_size = rpc_header.args_size();
    LOG(INFO) << "service_name: " << service_name << ", method_name: " <
method_name << ", args_size: " << args_size;
    // 读取参数
    std::string args_str;
    read_ok = coded_in.ReadString(&args_str, args_size);
    if (!read_ok) {
        LOG(ERROR) << "failed to read args_str";
        return;
    }
    // 查找服务对象
    auto it = service_map.find(service_name);
    if (it == service_map.end) {
        LOG(ERROR) << "service not found: " << service_name;
        return;
    }
    ServiceInfo& service_info = it->second;
    google::protobuf::Service* service = service_info.service;
    // 查找方法描述
    auto mit = service_info.m_method_map.find(method_name);
    if (mit == service_info.m_method_map.end) {
        LOG(ERROR) << "method not found: " << method_name;
        return;
    }
    const google::protobuf::MethodDescriptor* method = mit->second;
    // 创建请求和响应对象
    google::protobuf::Message* request = service->GetRequestPrototype(method)
.New();
    google::protobuf::Message* response = service->GetResponsePrototype(method
).New();
    if (!request->ParseFromString(args_str)) {
        LOG(ERROR) << "failed to parse request";
        return;
    }
    // 创建 Closure 回调
    google::protobuf::Closure* done = new muduo::net::BindArgs(
        this, conn, response);
    // 调用服务方法
    service->CallMethod(method, nullptr, request, response, done);
}

void KrpcProvider::SendRpcResponse(const muduo::net::TcpConnectionPtr& conn,
                                   google::protobuf::Message* response) {
    // 序列化响应
    std::string response_str;
    if (response->SerializeToString(&response_str)) {
        // 发送响应
        conn->send(response_str);
        LOG(INFO) << "response sent, size: " << response_str.size();
    } else {
        LOG(ERROR) << "failed to serialize response";
    }
    // 关闭连接
    conn->shutdown();
}

```
//ZkClient 封装的客户端类
class ZkClient {
public:
    ZkClient();
    ~ZkClient();
    // 启动连接
    void Start();
    // 创建 znode 节点
    void Create(const char* path, const char* data, int datalen, int state
= 0);
    // 根据指定路径获取 znode 节点值
    std::string GetData(const char* path);
private:
    // zk 的客户端会话句柄
    zhandle_t* m_zhandle;
};
```

封装ZooKeeper客户端

```
// RpcController 客户端控制类
class RpcController : public google::protobuf::RpcController {
public:
    // 构造函数
    RpcController();
    // 重置控制器状态，将失败状态和错误信息清空
    void Reset();
    // 判断当前RPC调用是否失败
    bool Failed() const;
    // 获取错误信息
    std::string ErrorText() const;
    // 设置RPC调用失败，并记录失败原因
    void SetFailed(const std::string &reason);
    // 未实现
    void StartCancel();

    bool IsCanceled() const;
    void NotifyOnCancel(google::protobuf::Closure* callback);
private:
    // RPC 方法执行过程中的状态
    bool m_failed;
    // RPC 方法执行过程中的错误信息
    std::string m_errText;
};
```

RPC控制类

```
// RpcChannel 客户端通道类
class RpcChannel
// 核心方法：实现 RpcChannel 的 CallMethod 接口
// 这是客户端发起 RPC 调用的入口
// 负责序列化请求、发送网络数据、接收响应
class RpcChannel : public google::protobuf::RpcChannel {
public:
    // 构造函数：接收一个 connection 来管理这条连接
    RpcChannel(bool connectNow);
    // 虚析构函数
    virtual ~RpcChannel() {}
    // RPC 调用的核心方法
    // 客户端发起一个远程过程调用时，框架会调用这个函数
    void CallMethod(const ::google::protobuf::MethodDescriptor* method,
                    ::google::protobuf::RpcController* controller,
                    const ::google::protobuf::Message* request,
                    ::google::protobuf::Message* response,
                    ::google::protobuf::Closure* done) override;
private:
    // 建立连接
    bool newConnect(const char *ip, uint16_t port);
    // 从 Zookeeper 查询服务地址
    std::string QueryServiceHost(ZkClient *zkclient,
                 std::string service_name, std::string method_name, int &idx);
private:
    // 存储客户端套接字
    int m_clientfd;
    std::string service_name;
    std::string method_name;
    std::string m_ip;
    uint16_t m_port;
    // 用于加引服务器 ip 和 port 的下标
    int m_idx;
};
```

```
// 构造函数：接收一个 connection 来管理这条连接
RpcChannel(bool connectNow);
// 虚析构函数
virtual ~RpcChannel() {}
// RPC 调用的核心方法
// 客户端发起一个远程过程调用时，框架会调用这个函数
void CallMethod(const ::google::protobuf::MethodDescriptor* method,
                ::google::protobuf::RpcController* controller,
                const ::google::protobuf::Message* request,
                ::google::protobuf::Message* response,
                ::google::protobuf::Closure* done) override;
private:
// 建立连接
bool newConnect(const char *ip, uint16_t port);
// 从 Zookeeper 查询服务地址
std::string QueryServiceHost(ZkClient *zkclient,
            std::string service_name, std::string method_name, int &idx);
private:
    // 存储客户端套接字
    int m_clientfd;
    std::string service_name;
    std::string method_name;
    std::string m_ip;
    uint16_t m_port;
    // 用于加引服务器 ip 和 port 的下标
    int m_idx;
};
```

客户端通道类

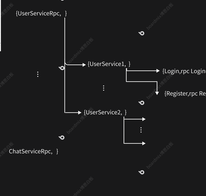
```
//SendRpcResponse回调函数
google::protobuf::Closure *done = google::protobuf::NewCallback<KrpcProvider,
                                google::protobuf::TcpConnectionPtr&,
                                google::protobuf::Message *>(this,
                                &KrpcProvider::SendRpcResponse,
                                conn, response);
```

NewCallback 是一个模板函数，它的功能是将一个成员函数绑定到特定的对象实例和参数上，生成一个可执行的 Closure 对象，模板参数含义：
- KrpcProvider: 目标接收，我们要绑定成员函数属于哪个类。（本例中是SendRpcResponse函数属于KrpcProvider类）
- this: 具体的对象实例。也就是说，当回调发生时，会调用 KrpcProvider 实例来执行任务。
- &KrpcProvider::SendRpcResponse: 这是指向成员函数的指针。它指定了当这条通道执行完（调用 done->Run()）时，真正要运行的函数。这个函数负责把 Response 序列化并通过网络发送。
- conn: (TcpConnectionPtr): 绑定的参数。它是了当前这条连接的指针。这是一个共享指针，确保连接在回调执行期间保持有效。
- response (Message): 绑定的参数。这是业务逻辑执行的结果数据。
这个函数绑定后的效果可以理解成这样的：
框架调用回调：当业务逻辑执行完毕，框架会通过调用它来执行 SendRpcResponse 函数，当前的 conn，以及这个空的 response 作为它的参数传递给函数。这条连接保存着Muduo对象后，在实现了发送数据的功能（比如调用Send()函数）后，会将这条连接保存的响应数据发送到response中。然后通过done->run()的方式通知框架回调函数。框架调用发送sendRpcResponse函数，整个过程等待这个回调函数完成。
google::protobuf::Closure* done，它实现了接口，它提供了一个 done->Run()的方法，框架调用发送done->run()调用发送响应函数SendRpcResponse回调函数。

```
// Message 部分代码
// 在框架上根据注册 RPC 服务，调用该 RPC 节点正在执行方法
service->CallMethod(method, nullptr, request, response, done);  // 调用服务方法
```

CallMethod是必须的，它的作用在于连接框架层与业务层的，实现框架层向业务层传递done对象的目的，在到达业务层的login方法执行完后，就会在done->Run()可以调用，回调发送数据至网络层，可以看到其实是参数传入了done。在之，CallMethod是多上（框架层）的下（业务层）的节点，对上，代表框架层已经把请求解析得明明白白了，对下，它是 UserService 业务代码的入口。
现在的逻辑是这样的：
- 框架层 (KrpcProvider)：
- 解析出 service_name, method_name。
- 查看对 request, response, done。
- 触发回调 (CallMethod) —— 跟整个代码的这一行
- 进入 Protobuf 生成的分发器。
- 进入业务层 (UserService)：
- 执行 Login 逻辑。
- 填充 response。
- 调用 done->Run()。
- 回到框架层 (KrpcProvider)：
- 执行 SendRpcResponse。
- 将 response 序列化并 conn->send。

这个架构本质上是一个**数据的传递**和**控制**。通过三层 “键-值” 对的映射，将一段模糊的网络二进制流，精确地定位到具体的 C++ 函数调用上。




```
void KrpcProvider::Run() {
    // 获取配置文件单例 KrpcConfig 实例 (假设)
    std::string ip = KrpcConfig::GetInstance().GetConfig().Load("rpcserverip");
    int port = 0;
    try {
        port = std::stoi(KrpcConfig::GetConfig().Load("rpcserverport"));
    } catch (...) {
        // 解析失败，使用默认端口
        port = 8000;
    }
    // 设置监听地址
    muduo::net::InetAddress address(ip, port);
    // 创建 TcpServer 对象
    std::shared_ptr<muduo::net::TcpServer> server =
std::make_shared<muduo::net::TcpServer>(&event_loop, address, "KrpcProvider");
    // 绑定连接回调和消息回调，以实现网络连接和数据接收
    server->setConnectionCallback(std::bind(&KrpcProvider::OnConnection, this,
std::placeholders::_1));
    server->setMessageCallback(std::bind(&KrpcProvider::OnMessage, this,
std::placeholders::_1, std::placeholders::_2, std::placeholders::_3));
    // 设置muduo的线程数量
    server->setThreadNum(4);
    /**
    * @brief zookeeper 服务注册：将 service_name 和 method_name 注册到 zookeeper 服务器
    */
    // 开始监听RPC请求之前，先向ZooKeeper注册服务 Zookeeper 上，以 RPC 客户端可以从 Zookeeper 上发现服务
    ZkClient zkclient;
    zkclient.Start();  // 连接zookeeper 服务器
    // server_name 为命名空间，method_name 为临时节点
    for auto kpp : service_map {
        // service_name 在 zookeeper 中的目录路 "/" + service_name (永久节点)
        std::string service_path = "/" + kpp.first;
        char data[128] = {0};
        int len = sprintf(data, "%s:%d", ip.c_str(), port);
        // method_name 在 zookeeper 中的目录路 "/" + service_name/method_name (临时节点)
        auto const& methods = kpp.second.m_method_map;
        for(auto mp : methods) {
            std::string method_path = service_path + "/" + mp.first;
            char method_path_data[128] = {0};
            sprintf(method_path_data, "%s:%d", ip.c_str(), port);
            // ZOO_EPHEMERAL 表示这个节点是临时节点，会客户端断开连接后，Zookeeper会自动删除这个
            int flag = ZOO_EPHEMERAL;
            int state = zkclient.Create(method_path.c_str(), method_path_data,
strlen(method_path_data), ZOO_EPHEMERAL);
        }
    }
    // RPC 服务端准备启动，打印信息
    std::cout << "RpcProvider start service at ip: " << ip << ":" << port <<
std::endl;
    // 启动网络服务
    server->start();
    event_loop.loop();  // 阻塞事件循环
}
```

Run()是由RPC服务节点
1. 读取配置文件 (RPC 服务器 IP 和 端口)；
2. Muduo引擎实例化服务器监听和连接回调函数 server;
3. Zookeeper服务注册(kv store);
4. 启动事件循环 loop


```
void LoginServiceImpl::CallMethod(const ::google::protobuf::MethodDescriptor*
method,
                                  ::google::protobuf::RpcController* controller,
                                  const ::google::protobuf::Message* request,
                                  ::google::protobuf::Message* response,
                                  ::google::protobuf::Closure* done) {
    // 调用具体的 Login 实现
    Login(dynamic_cast<const ::google::protobuf::LoginRequest*>(request),
          dynamic_cast<::google::protobuf::LoginResponse*>(response));
    // 执行完成后，调用 done->Run()
    done->Run();
}
```

可以看到，Login函数的参数，确实是google::protobuf::Closure*对象的done。
因此业务层函数实现需要接收来自框架层的Send对象(形参的参数中的response不是指向Send对象的，而是指向框架了，NewCallback参数中的response已经被转换为done对象了。

最终心的调用关系图

```
【调用顺序流程】 → request ←—— 你的业务函数
```

设置对应客户端，调用RpcCallMethod成员函数重载产编的核心调用方法，该成员函数主要包括流程如下：
1. 把客户端请求（服务名 + 方法名 + 请求数据）序列化发送到服务端，并且等待响应并反序列化解析response；
2. 客户端连接 ZooKeeper 服务器，并查询获取目标服务地址；
3. 建成连接 RPC 服务器；
4. 序列化 RPC 请求为 send_rpc_str（包括请求头和参数）；
5. send() 发送 RPC 请求至 S；
6. recv() 接收服务器的 RPC 响应至 recv_buf；
7. 反序列化响应内容到 response 对象；
8. closein_clientfd() 关闭连接